

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a2ed8791b47b2e94a.jsonl`  
- SHA-256: `11da0214764b70e64c6853f4c8685bf1f2bf52b9415155036721d384dce1fe10`  
- Source modified: `2026-06-13T19:15:39+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:13:53.202Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title": "_person_features falls back for frame_width but NOT frame_height ? fh can stay 0.0 and silently half-zero the feature vector", "severity": "low", "file": "backend/pipeline/segmenters/fusion_hybrid.py:99-100", "detail": "In _person_features: `fw = det[\"frame\_width\"]` or `frame\_width` (falls back to the probed width when cv2 reports 0) but `fh = det[\"frame\_height\"]` or 0.0` (NO fallback ? defaults to 0.0). If detections_per_frame returns frame_height=0.0 (cv2 can fail to read CAP_PROP_FRAME_HEIGHT on some codecs/containers; person_detector.py:detections_per_frame returns frame_height=0.0 on a failed-open and passes through the raw cap value otherwise), then fh=0.0 is fed into compute_person_features. I confirmed the effect: with fw=1280, fh=0.0 the vector becomes {players_in_court:2, two_sided_motion:0, mean_player_motion:0, in_court_ratio:1, shuttle_player_colocation:0} ? i.e. the three height-dependent features collapse to 0 while players_in_court/in_court_ratio (height-independent when court_polygon is None) report normal values. That is a misleading PARTIALLY-degraded feature row rather than a clean all-zero/skip, and it would silently poison the P3 training set for any clip where cv2 can't read the height. The width path is protected; the height path is not.", "fix": "Make the height fallback symmetric. Either `fh = det[\"frame\_height\"]` or `frame\_height` (but the engine only probes width via _probe_fps_width, so there is no probed height to fall back to), or better: probe height alongside width, or skip the person-cue enrichment for the window when det['frame_width'] <= 0 or det['frame_height'] <= 0 (return an empty/zeroed dict consistently) so a row is never half-computed."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:57.351Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:13:58.233Z)

```

1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13     never imports torch / touches the GPU.
14     - ``_select_for_handoff`` ? optional served Gate A/B: when
15     ``indexing.fusion.min_crossings``
16     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17     are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
20     experiment
21     harness can re-score any variant.
22
23     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24     but
25     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26     seeds
27     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28     default
29     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30     through the
31     experiment harness (EXPERIMENT_HARNESS.md), never silently.
32     """
33     from typing import Any, Dict, List, Optional, Tuple
34
35     from backend.pipeline.segmenters.base import segmenter_registry
36     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37     from backend.pipeline.detectors.base import DetectorRunner
38     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40     from backend.pipeline.detectors import rally_gate
41     from backend.pipeline.detectors import fusion_features as ff
42
43     class FusionSegmenter(TrajectoryHybridSegmenter):
44         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45         B)."""
46
47         family = "fusion"
48         display_label = "Fusion"
49
50         def __init__(self, db: Any, config: Any):
51             super().__init__(db, config)
52             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53             default path).
54             self._person: Optional[Any] = None
55
56         @property
57         def name(self) -> str:
58             return "fusion_hybrid"
59
60         @property
61         def description(self) -> str:
62             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
63             rallies, "
64
65             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,

```

[illegible]

```

109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                             window_stats: List[Dict[str, Any]],
111                             idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. user (2026-06-13T19:13:58.699Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12    - `per_frame` is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's `frame` ? no clip re-
timing).
14    - A shuttle point has `frame`, `visible`, `x`, `y` (pixels) ? the WASB
trajectory shape.
15    - `court_polygon` is normalised 0-1 `(x, y, ...)` or `None` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17    - `net` is `(axis 'x'|'y', position 0-1 normalised)` or `None` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except `mean_player_motion` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])

```

```

-> bool:
31     """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32     (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33     if polygon is None or len(polygon) < 3:
34         return False
35     x, y = point
36     inside = False
37     n = len(polygon)
38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                        frame_height: float,
78                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79         """Per-frame count of in-court persons (one entry per sampled frame)."""
80         return [
81             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
82             for _frame_idx, dets in sorted(per_frame.items())
83         ]
84
85
86     def _median(vals: Sequence[float]) -> float:
87         if not vals:
88             return 0.0
89         s = sorted(vals)
90         n = len(s)

```

```

91         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
101 flicker)."""
102         if not counts:
103             return 0.0
104         return sum(1 for c in counts if c >= min_players) / len(counts)
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109 by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                         frame_height: float, net: Optional[Net],
128                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131 A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132 The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133 0.0.
134 """
135         if net is None or not per_frame:
136             return 0.0
137         axis, pos = net
138         both = 0
139         for _frame_idx, dets in per_frame.items():
140             near, far = False, False
141             for _tid, box in dets:
142                 if not person_in_court(box, frame_width, frame_height, court_polygon):
143                     continue
144                 fx, fy = _foot_norm(box, frame_width, frame_height)
145                 coord = fx if axis == "x" else fy
146                 if coord < pos:
147                     near = True
148                 else:
149                     far = True
150             if near and far:
151                 both += 1
152         return both / len(per_frame)
153

```

```

154     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
155                        pad_frac: float) -> bool:
156         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157         width/height (the 'adjacent / in-hand' tolerance)?"""
158         x1, y1, x2, y2 = box
159         px = pad_frac * frame_width
160         py = pad_frac * frame_height
161         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                                frame_width: float, frame_height: float,
166                                pad_frac: float = 0.03) -> float:
167         """Fraction of (visible shuttle frames that were also person-sampled) in which the
168         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
169         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
170         """
171         if not per_frame or frame_width <= 0 or frame_height <= 0:
172             return 0.0
173         aligned = 0
174         colocated = 0
175         for p in shuttle_points or []:
176             if not getattr(p, "visible", False):
177                 continue
178             dets = per_frame.get(int(p.frame))
179             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
180                 continue
181             aligned += 1
182             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183                    for _tid, box in dets):
184                 colocated += 1
185         return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                frame_width: float, frame_height: float,
198                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                net: Optional[Net] = None,
200                                min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }

```

5. assistant (2026-06-13T19:14:02.206Z)

Now let me verify the key claims. Let me read the `person\_detector.py` `detections\_per\_frame` method to confirm what it returns for `frame\_height`.

6. user (2026-06-13T19:14:03.528Z)

```
diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

    return windows
+
+ def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                          frame_skip: int = 3) -> Dict[str, Any]:
+     """Per-frame person detections over `[start_frame, end_frame]` of the ORIGINAL video.
+
+     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+     `extract_action_windows_from_chunk` (which detects ITS OWN windows and discards the
+     per-frame boxes), this returns the raw per-frame detections so the fusion features can
+     align them with the shuttle trajectory. It reads the source by **absolute frame index**
+     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+     the WASB trajectory's `.frame` exactly ? that alignment is what
+     `fusion_features.shuttle_player_colocation` / `two_sided_motion` rely on.
+
+     Returns `{ "frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+     "frame_height"}`. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+     accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+     treat the indices as ±a few frames on real footage ? fine for window-level features.
+     """
+     self.lazy_load_model()
+     cap = cv2.VideoCapture(video_path)
+     if not cap.isOpened():
+         logger.error(f"Could not open {video_path}")
+         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+     fps = cap.get(cv2.CAP_PROP_FPS)
+     if fps <= 0:
+         fps = 30.0
+     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+     step = max(frame_skip, 1)
+     tracker = self.make_tracker(fps / step)
+
+     start_frame = max(0, int(start_frame))
+     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+     frame_idx = start_frame
+     while frame_idx <= end_frame:
+         ret, frame = cap.read()
+         if not ret:
+             break
+         if (frame_idx - start_frame) % step == 0:
+             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+             frame_idx += 1
+     cap.release()
+     return {"frames": frames, "fps": fps,
+           "frame_width": float(frame_width), "frame_height": float(frame_height)}
```

7. assistant (2026-06-13T19:14:10.393Z)

Now I have confirmed the `detections\_per\_frame` implementation. Let me analyze the claims carefully.

Confirmed so far:

1. `person\_detector.py:detections\_per\_frame` returns `frame\_height=0.0` on a failed-open (line: `return {"frames": {}, "fps": 30.0, "frame\_width": 0.0, "frame\_height": 0.0}`). On success it returns `float(cap.get(cv2.CAP\_PROP\_FRAME\_HEIGHT))`.
2. `fusion\_hybrid.py:99-100`: `fw = det["frame\_width"] or frame\_width` (fallback) vs `fh = det["frame\_height"] or 0.0` (no real fallback ? `or 0.0` is a no-op).

Now let me critically examine the finding's central claim about the asymmetry mattering. The key question: ****on a failed-open, both `frames={}` AND `frame\_width=0.0`****. So on a failed-open, `det["frames"]` is empty. Let me verify what `compute\_person\_features` produces with an empty `per\_frame` dict, regardless of `fh`.

8. user (2026-06-13T19:14:11.408Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

```
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index b2847c9..149282e 100644
```

```
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
     except (ValueError, TypeError):
         return max(3, int(duration / 0.8))
```

```
+ # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -----
```

```
+ def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                             frame_width: float, video_path: str,
+                             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
+     """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
+     returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+     rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+     features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+     return {
+         "peak_velocity": cw["peak_velocity"],
+         "mean_velocity": cw["mean_velocity"],
+         "active_frame_count": cw["active_frame_count"],
+         "track_id_count": cw["track_id_count"],
+     }
```

```
+ def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                          window_stats: List[Dict[str, Any]],
+                          idx_cfg: Dict[str, Any]) -> List[int]:
+     """Indices of the candidate windows that proceed to the AI handoff (and thus may
+     become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+     ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+     Persisted candidates are NOT affected ? they remain the full superset for offline
+     re-scoring."""
+     return list(range(len(windows)))
```

```
+ def process_video(self, video_id: str, video_path: str, # type: ignore[override]
+                   player_pool: Optional[List[str]] = None,
+                   max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
```

```
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
```

```
     f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
     f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
```

```
+ # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
```

```

+         # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+         # persistence and the handoff gate below.
+         window_stats = [
+             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+             for cw in all_candidate_windows
+         ]
+
+         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
+         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
+         if store_candidates:
+             for i, cw in enumerate(all_candidate_windows):
+                 stats = {
+                     "peak_velocity": cw["peak_velocity"],
+                     "mean_velocity": cw["mean_velocity"],
+                     "active_frame_count": cw["active_frame_count"],
+                     "track_id_count": cw["track_id_count"],
+                 }
+                 candidate_ids[i] = self.db.add_candidate_segment(
+                     video_id, detector=self.name,
+                     start_time=cw["start"], end_time=cw["end"],
+                     chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
+                     stats=stats, detection_params=detection_params,
+                     stats=window_stats[i], detection_params=detection_params,
+                 )
+
+         if not all_candidate_windows:
+             return []
+
+         # --- Phase 3: AI provider handoff over padded candidate clips ---
+         # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+         # candidates above stay the full superset ? only the served play_segments are gated.
+         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
+ idx_cfg)
+         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
+         os.makedirs(handoff_dir, exist_ok=True)
+         logger.info("Phase 3: %s validation on %d candidate windows...",
+                     provider_name, len(all_candidate_windows))
+
+         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
+             w_start = max(0, window["start"] - handoff_padding)
+
+         @@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
+
+             ai_segments: List[dict] = []
+             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
+                 futures = [executor.submit(process_candidate_window, wi, cw)
+                     for wi, cw in enumerate(all_candidate_windows)]
+                 futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                     for wi in handoff_indices]
+                 for future in futures:
+                     ai_segments.extend(future.result())
+             try:

```

9. assistant (2026-06-13T19:14:20.111Z)

Now let me trace through the exact scenario the finding claims. The finding's central claim is a scenario where `fw=1280, fh=0.0` but `det["frames"]` is non-empty. Let me carefully verify whether such a scenario is actually reachable.

The critical insight: In `detections\_per\_frame`, on the **failed-open** path, BOTH `frames={}` and `frame\_width=0.0` AND `frame\_height=0.0` are returned together. So:

- If open failed ? `frames={}`, `fw = 0.0` or `1280 = 1280`, `fh = 0.0` or `0.0 = 0.0`. But `per\_frame={}` empty.
- On **success** ? `frame\_width` and `frame\_height` are BOTH `float(cap.get(...))`. The finding claims cv2 can read width but not height on some codecs, giving `fw>0, fh=0` with non-empty

frames.

Let me test the actual behavior of `compute\_person\_features` with both scenarios to verify the finding's claimed feature vector.

10. user (2026-06-13T19:14:30.200Z)

```
=== fw=1280, fh=0.0 (the claimed half-zero case) ===
  players_in_court: 2.0
  two_sided_motion: 0.0
  mean_player_motion: 0.0
  in_court_ratio: 1.0
  shuttle_player_colocation: 0.0
=== fw=1280, fh=720 (healthy case) ===
  players_in_court: 2.0
  two_sided_motion: 0.0
  mean_player_motion: 0.0078125
  in_court_ratio: 1.0
  shuttle_player_colocation: 0.0
=== fw=0.0, fh=0.0 with EMPTY per_frame (failed-open case) ===
  players_in_court: 0.0
  two_sided_motion: 0.0
  mean_player_motion: 0.0
  in_court_ratio: 0.0
  shuttle_player_colocation: 0.0
=== fw=1280, fh=0.0 WITH court_polygon set ===
  players_in_court: 0.0
  two_sided_motion: 0.0
  mean_player_motion: 0.0
  in_court_ratio: 0.0
  shuttle_player_colocation: 0.0
```

11. assistant (2026-06-13T19:14:40.394Z)

This confirms the finding's claimed feature vector exactly. With `fw=1280, fh=0.0` and a non-empty `per\_frame` (and `court\_polygon=None`), the vector is:

- `players\_in\_court: 2.0` (height-independent ? normal)
- `in\_court\_ratio: 1.0` (height-independent ? normal)
- `two\_sided\_motion: 0.0`, `mean\_player\_motion: 0.0`, `shuttle\_player\_colocation: 0.0` (height-dependent ? collapsed)

So the partially-degraded row IS reproducible. The remaining question is whether the **trigger** is real: can `detections\_per\_frame` actually return `frame\_width > 0, frame\_height = 0.0` with a non-empty `frames` dict?

Let me look at the success path carefully. On success: `frame\_width = cap.get(cv2.CAP\_PROP\_FRAME\_WIDTH)` and `frame\_height = cap.get(cv2.CAP\_PROP\_FRAME\_HEIGHT)`. There's no validation/clamp on either. The `while` loop reads frames via `cap.read()` and populates `frames`. So if cv2 opens the file, reads frames successfully, but returns `CAP\_PROP\_FRAME\_HEIGHT = 0` while `CAP\_PROP\_FRAME\_WIDTH > 0`, we get exactly the partial case.

Let me check whether there's any precedent in the codebase for cv2 returning 0 for these properties (to assess the plausibility of the trigger), and confirm the width-fallback rationale.

12. user (2026-06-13T19:14:41.955Z)

```
backend\pipeline\detectors\person_detector.py:162:         frame_width =
cap.get(cv2.CAP_PROP_FRAME_WIDTH)
backend\pipeline\detectors\person_detector.py:282:         frame_width =
cap.get(cv2.CAP_PROP_FRAME_WIDTH)
backend\pipeline\detectors\person_detector.py:283:         frame_height =
cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
tests\test_person_detector.py:21:         if prop == cv2.CAP_PROP_FRAME_WIDTH:
```

```
backend\pipeline\segmenters\trajectory_hybrid.py:62:         width =
cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
tests\test_yolo_hybrid.py:22:         if prop == cv2.CAP_PROP_FRAME_WIDTH:
```

13. user (2026-06-13T19:14:43.166Z)

```
58:     def _probe_fps_width(video_path: str) -> tuple:
59:         """Return (fps, frame_width) for a video, with sensible fallbacks."""
83:             frame_width: float, video_path: str,
144:         fps, frame_width = self._probe_fps_width(video_path)
187:         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
204:         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
```

14. assistant (2026-06-13T19:14:46.056Z)

Let me look at `\_probe\_fps\_width` to confirm the finding's claim that the engine only probes width (no probed height to fall back to).

15. user (2026-06-13T19:14:46.832Z)

```
56
57     @staticmethod
58     def _probe_fps_width(video_path: str) -> tuple:
59         """Return (fps, frame_width) for a video, with sensible fallbacks."""
60         cap = cv2.VideoCapture(video_path)
61         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63         cap.release()
64         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66     @staticmethod
67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
87         BASE
```

16. assistant (2026-06-13T19:14:56.483Z)

This fully confirms the finding's assertions:

1. **`\_probe\_fps\_width` only probes width** (line 58-64), with fallback `width if width > 0 else 1280.0`. There is NO probed height ? confirmed.
2. **`frame\_width` flows into `\_person\_features`** as the engine's probed-width fallback. So `fw = det["frame\_width"]` or `frame\_width` recovers a sensible width even if cv2 reports 0.
3. **`fh = det["frame\_height"]` or 0.0`\*\* has no such recovery ? `or 0.0` is a no-op, since `0.0 or 0.0 == 0.0`.

Now let me critically assess the **trigger plausibility** ? the crux of whether this is real. The finding claims cv2 can return `frame_width > 0`` and `frame_height == 0`` independently. Let me check the existing test mocks and consider the realistic failure modes.

17. user (2026-06-13T19:14:57.964Z)

```
275         if not cap.isOpened():
276             logger.error(f"Could not open {video_path}")
277             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
278
279         fps = cap.get(cv2.CAP_PROP_FPS)
280         if fps <= 0:
281             fps = 30.0
282         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
283         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
284         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
285         step = max(frame_skip, 1)
286         tracker = self.make_tracker(fps / step)
287
288         start_frame = max(0, int(start_frame))
289         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
290         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
291         frame_idx = start_frame
292         while frame_idx <= end_frame:
293             ret, frame = cap.read()
294             if not ret:
295                 break
296             if (frame_idx - start_frame) % step == 0:
297                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
298             frame_idx += 1
299         cap.release()
```

18. user (2026-06-13T19:14:58.842Z)

```
tests\test_fusion_hybrid.py:100:     fake_pd.detections_per_frame.return_value = {
tests\test_fusion_hybrid.py:102:         "fps": 30.0, "frame_width": 1000.0, "frame_height":
1000.0,
tests\test_fusion_hybrid.py:109:     fake_pd.detections_per_frame.assert_called()           # the
person detector WAS used
```

19. assistant (2026-06-13T19:15:15.312Z)

I have confirmed all the load-bearing facts. Let me now make my adversarial assessment.

****Adversarial scrutiny of the trigger:****

The finding hinges on cv2 returning `CAP_PROP_FRAME_WIDTH > 0`` while `CAP_PROP_FRAME_HEIGHT == 0`` (or vice-versa, but the asymmetry only bites when height is the one that's 0). Is this realistic?

- The two properties are read independently via separate `cap.get()` calls on the same `VideoCapture``. There is no validation that couples them.
- In practice, when cv2 can read one dimension it almost always reads both ? they come from the same stream metadata header. A scenario where width parses but height returns exactly 0 is unusual. The more common failure mode (corrupt/unreadable container) makes the open fail entirely (caught by `isOpened()`, returns all-zero + empty frames ? clean all-zero row) OR makes BOTH props return 0.
- If BOTH return 0 but frames still read (some streaming/codecs cases where props are unavailable but decode works): then `fw = 0.0` or `frame_width = 1280`` (probed fallback, possibly wrong aspect) and `fh = 0.0` or `0.0 = 0.0`` (collapses). This is actually the **more realistic** trigger ? and the asymmetry still bites: width gets a fallback, height gets zero. So the finding's mechanism holds even if its specific "width OK, height 0" framing is the rarer sub-case.

So the asymmetry is genuinely there and the degraded-row outcome is real and reproducible. The probability of the trigger firing is **low** (cv2 usually reads both dims or neither-with-failed-open), which matches the "low" severity. The consequence ? a silently half-zeroed feature row poisoning the P3 training set rather than a clean all-zero/skip ? is real and correctly described.

Counter-argument considered: Is `fh` even used meaningfully if `fw` is also wrong? When both props fail, `fw` falls back to a possibly-wrong 1280, so width-normalized features are ALSO slightly off in that path. But the finding's specific complaint ? that the height path lacks the protection the width path has, producing an inconsistent/asymmetric degradation ? is accurate and is a real code-quality/data-quality defect. The fix proposal (probe height too, or skip enrichment when either dim ? 0 to return a consistent row) is sound and matches the codebase's stated "degrades gracefully / clean zero" discipline in `fusion_features.py`'s module docstring.

This is a real, correctly-reasoned, low-severity finding. The code, the trigger source, and the reproduced feature vector all check out. It is not a nitpick ? it concerns silent training-data corruption ? though its real-world hit rate is low.

20. user (2026-06-13T19:15:30.425Z)

Structured output provided successfully

21. assistant (2026-06-13T19:15:38.947Z)

I have completed the adversarial verification of this finding.

Verdict: REAL (low severity).

The finding holds against the actual code. Key confirmations:

- `backend/pipeline/segmenters/fusion_hybrid.py:99-100` ? the asymmetry is exactly as described: `fw = det["frame_width"] or frame_width` (real probed-width fallback) vs `fh = det["frame_height"] or 0.0` (no-op; `0.0 or 0.0 == 0.0`).
- `backend/pipeline/detectors/person_detector.py:282-283` ? width and height come from independent, unvalidated `cap.get()` calls; height can be 0 on the success path.
- `backend/pipeline/segmenters/trajectory_hybrid.py:57-64` ? `_probe_fps_width` only probes width (fallback 1280.0); there is genuinely no probed height to fall back to.
- I reproduced the claimed half-zero vector by executing `compute_person_features` with `fw=1280, fh=0.0`: `{players_in_court:2.0, two_sided_motion:0.0, mean_player_motion:0.0, in_court_ratio:1.0, shuttle_player_colocation:0.0}` ? the three height-dependent features collapse while the two height-independent counts report normal values.

Adversarial caveats that justify the low severity: the failed-open path returns both `frame_width=0.0` and an empty `frames` dict, so it yields a *clean* all-zero row ? the misleading half-zero row only arises on a successful open where cv2 decodes frames but reports height 0, which is an uncommon cv2 state. The defect is real (silent partial corruption of the P3 training set, inconsistent with `fusion_features.py`'s stated graceful-degradation contract) but has a low real-world hit rate.